UNIDADE CURRICULAR: PROJETO APLICADO I

EMPRESA PARCEIRA: SCHULZ S.A.

DOCENTE: PROF. MSC. HUGO MENEZES BARRA e PROF. RODRIGO LOSINA

ROTEIRO DE INSTRUÇÕES DE ENTREGA — AV 03 (TEMA 03)

Construção do MVP (Parte 1): Implementação das Classes do Sistema em Programação Orientada a Objetos (POO em Java ou Python) e Conexão com Banco de Dados Relacional (SQLite)

1. Contextualização da Entrega 03

Nas etapas anteriores (AV 01 e AV 02), a equipe atuou na investigação dos processos manuais da Schulz S.A., no levantamento de requisitos operacionais, regras de negócio e na organização e modelagem relacional do banco de dados SQLite (composto pelas tabelas tecnicos, equipamentos e medicoes).

Nesta terceira entrega (AV 03 — Construção do MVP - Parte 1), o grupo evoluirá da modelagem conceitual para a construção efetiva do código do sistema. O objetivo central é implementar a arquitetura em Programação Orientada a Objetos (POO) em linguagem Java ou Python, construindo as classes de domínio, os objetos de acesso a dados e os algoritmos de validação de regras de negócio metrológicas e manipulação do banco relacional SQLite.

2. Requisitos Técnicos e Arquitetura do Sistema

O código fonte pode ser desenvolvido na linguagem de preferência do grupo (Java ou Python), estruturado dentro do ambiente VS Code (ou IDE de escolha) e versionado em repositório GitHub. A aplicação deve seguir boas práticas de engenharia de software, incluindo encapsulamento, separação de responsabilidades e tratamento de dados.

2.1 Estrutura de Classes Esperada (Orientação a Objetos)

1. Classe Tecnico: Representa a entidade do profissional calibrador (`id_tecnico`, `matricula`, `nome`, `setor`, etc).
2. Classe Equipamento: Representa os instrumentos de medição (`id_equipamento`, `codigo_tag`, `descricao`, `fabricante`, etc).
3. Classe Medicao: Encapsula o ensaio de calibração, contendo a lógica de validação do status (‘APROVADO`/`REPROVADO’) com base nas tolerâncias e no valor obtido.
4. Classe de Conexão e Persistência: Responsável por executar os comandos SQL

('INSERT', 'SELECT', 'UPDATE') e gerenciar a conexão JDBC (Java) ou 'sqlite3' (Python) com o SQLite.

3. Entregáveis Requisitados (Detalhamento dos Arquivos)

Item Requisitado	Conteúdo Esperado (Execução no VS Code)	Extensão / Formato
1. Classes de Domínio e Conexão SQL (04_classes.py ou .java)	Implementação das classes POO (Tecnico, Equipamento, Medicao) em Java ou Python com atributos, métodos de validação de tolerância e métodos CRUD para comunicação com SQLite (via JDBC em Java ou 'sqlite3' em Python).	Arquivo de Código (.java ou .py)
2. Script / Classe Principal de Demonstração (05_main.py ou Main.java)	Script com a Classe principal demonstrando a instanciação dos objetos, inserção de registros de testes no banco SQLite, execução de consultas relacionais com JOIN e exibição do relatório de calibração no terminal.	Arquivo de Código (.java ou .py)
3. Documentação Técnica do MVP - Parte 1 (06_mvp_parte1.md)	Documento explicativo abordando os padrões de POO utilizados na linguagem escolhida (Java ou Python), diagramação textual/UML das classes, tratamento de erros SQL e validação do fluxo funcional do MVP.	Arquivo Markdown (.md)

4. Regras de Formato e Prazos de Entrega

INFORMAÇÕES DE SUBMISSÃO E PRAZOS

• Linguagens Aceitas: Java ou Python.
• Formato de Envio: Arquivo compacto (.ZIP) postado no Ambiente Virtual de Aprendizagem (AVA).
• Conteúdo do ZIP: Documento PDF explicativo consolidado + Link do repositório GitHub atualizado contendo os arquivos solicitados.
• Data Limite de Entrega: 03/11/2026
• Critérios de Avaliação: Qualidade da modelagem POO na linguagem escolhida (Java ou Python), correto tratamento de dados com o banco SQLite, alinhamento aos requisitos do projeto e organização do código.